ChatGPT

Detecting Intentions Using LLMs

Large Language Models (LLMs) have recently been leveraged to **recognize and infer intentions** across various domains. In task-oriented **dialogue systems**, for example, prompting techniques (like adaptive chain-of-thought prompting) enable LLMs to classify user intents more accurately than traditional intent classifiers [1]. Arora *et al.* (EMNLP 2024) show that by using in-context examples and reasoning steps, GPT-based models can achieve near state-of-the-art intent detection performance with minimal training data [2]. Beyond dialogue, researchers are exploring LLMs for **plan/goal recognition**. In robotics and autonomous agents, LLMs can interpret commands and **decompose high-level goals into subgoals** hierarchically. For instance, the ReAcTree framework dynamically builds an *"agent tree"* of subgoals and control-flow nodes from a natural language instruction, effectively modeling a hierarchy of intentions that an agent should fulfill [3]. Each node in the tree represents a sub-intent (subgoal), and the LLM can decide to expand nodes into further subgoals when tasks are complex [3] [4]. This hierarchical reasoning helps prevent error propagation and makes long-horizon tasks more tractable for LLM planners.

Another emerging use of LLMs is to **infer the implicit intent behind software-related inputs**. When a user gives an instruction to a coding assistant (e.g. *"Add a function to do X"*), the **LLM must discern the underlying intent** – what functionality is truly desired. Recent work has shown that prompting an LLM to explicitly reason about intent before generating code can significantly improve results [5]. For example, the approach *"Your Coding Intent is Secretly in the Context"* frames code generation as a three-stage process: first, the model analyzes the preceding code context to **infer the developer's intent** (by extracting clues about desired functionality), then optionally lets the user refine this inferred intent, and finally generates the code according to that intent [5]. By guiding the LLM through step-by-step extraction of intent signals from context, this method achieved over *20%* relative gains in code generation accuracy on benchmarks [6]. In essence, the LLM serves as an **intent detector** before it becomes a code generator.

LLMs are also being applied to **detect intentions in code itself**. Because LLMs are trained on vast code corpora, they possess a kind of *"code comprehension"* ability. This has been exploited in static analysis tasks – for example, identifying the *intentions behind resource-management code*. A 2024 study introduced *InferROI*, which prompts an LLM to recognize *"resource-oriented intentions"* in a given code snippet (such as the intent to acquire a resource, release it, or check its availability) [7]. These inferred intents are then converted to formal specifications and checked against control-flow paths to detect bugs like resource leaks. Impressively, InferROI's LLM-driven intent inference reached ~81.8% recall in identifying resource-handling intents in Java code, and helped catch resource leak bugs that traditional analyzers missed [8] [7]. This demonstrates how LLMs can **uncover the intent behind specific code patterns** (in this case, the purpose of certain API calls or code blocks) and use that understanding to improve software analysis.

Overall, **using LLMs for intent detection** is a high-impact research area spanning NLP and software engineering. From classifying user intents in conversations to probing code context for developer intent, LLMs provide a powerful tool to infer "what is wanted" or "what was intended" without explicit annotations. Techniques like reasoning-based prompting, interactive query of the user, and combining LLM insights with traditional analysis are proving effective in bridging the gap between *what humans intend* and *what the AI or code does*. Next, we delve into what kinds of intentions exist inside software and how they are modeled.

Types of Intentions Inside Software

When we talk about "intentions inside software," we refer to the **goals, purposes, or rationales** that software artifacts encode. Modern research has identified several levels at which such intentions can be categorized and extracted:

- **Commit Change Intent (Maintenance Activities):** At the level of version control, every code change (commit) is made for a reason. Common **intent categories** include fixing bugs (corrective maintenance), adding new features or capabilities (adaptive/perfective maintenance), refactoring or improving code structure, updating documentation, etc. In fact, most research on commit intent focuses on classifying the commit by its *maintenance activity type* [9]. Knowing this intent is useful because maintenance consumes up to 50–90% of project effort [10]. However, developers rarely document the true intent of each commit; commit messages are often incomplete [11] [12]. To address this, recent approaches use ML and code analysis to **infer commit intent from the code changes themselves**. For example, Heričko *et al.* (2024) embed code diffs with a transformer-based model (CodeBERT/GraphCodeBERT) and achieve state-of-the-art accuracy in classifying commit intent (bug-fix vs. feature addition, etc.) [13] [14]. This means the *"why"* behind a change – often only hinted at in text – can be predicted from the *"what"* in the code. Such intent classification enables better tracing of software evolution (e.g. identifying if an unexpected number of feature-add commits occur during a supposed bug-fixing phase [15]).

- **Functional Intent in Code:** At a finer granularity, **individual code snippets or functions carry intent** – they implement some intended behavior or requirement. For instance, a function might have the intent to *"parse user input"*, *"compute a discount"*, or *"validate a file handle"*. Even without documentation, this intent can sometimes be inferred from context or patterns. LLM-based techniques are being used to recover these implicit intents. The *"Your Coding Intent..."* framework mentioned earlier is one example: it deduces a function's intended purpose from clues in preceding code and generates an explanatory docstring capturing that intent [16] [17]. Another example is **resource management intent**: InferROI's LLM identifies code segments meant for acquiring or releasing resources [7]. By formalizing such intents, tools can detect discrepancies (e.g., a missing release where one was intended). More generally, researchers are looking at how code comment generation can include *why* the code exists (the developer's rationale) rather than just *what* it does. Capturing this functional intent is crucial for code comprehension and documentation.

- **Developer Requirements and Goals:** On a higher level, software is created to fulfill **stakeholder requirements** or user stories – essentially, the *intentions of users or designers*. In requirements engineering, these are often modeled as **goals** or **intentions** of various actors. Classic frameworks like **i** *(i-star) and goal-oriented requirement analysis explicitly represent these intents and their decomposition. A recent meta-model called I-Tree (Intention Tree) builds on this idea by structuring stakeholder objectives into a tree of intentions and sub-intentions [18]. In an I-Tree, a high-level user intention (e.g. "manage personal finances") can be decomposed into more specific intentions ("track expenses", "generate budget report", etc.), and constraints or relationships between them are noted [18]. Each node in the tree corresponds to a functional requirement (a capability the software should have) – effectively, the software's intended behaviors** in a hierarchical view. While this is more about design-time intentions, it provides a foundation: even code and commits ultimately trace back to these higher-level intended functionalities.

- **Rationale in Code History:** Another sort of "intention inside software" is the **rationale recorded in development artifacts** like issue descriptions, design discussions, or commit messages. These often describe *why* a change was made ("improve response time by caching X" or "fix null pointer exception in Y"). Mining such data can reveal patterns of intent, such as performance optimization, security hardening, usability improvement, etc. For example, research in mining software repositories might cluster commits by intent (all performance-related changes versus all bug-fixes). Some studies even try to generate commit messages from diffs (effectively articulating the intent of a code change in natural language) using LLMs [11] [19]. As LLMs get better at understanding both code and natural language, they can bridge the two – e.g. **inferring the likely intent from a code diff and producing a summary** ("Refactor loop to reduce memory usage"). This helps in cases where intent wasn't explicitly documented by humans.

In summary, **software intentions range from fine-grained to coarse-grained**: from a single line of code's purpose (e.g. "open file for reading") up to an entire system's goal ("provide secure messaging service"). Recent research is tackling intention inference at all these levels. The highest-impact work combines static or historical analysis with LLMs' semantic understanding – enabling tools that not only *detect* what a developer or user intended, but also categorize those intentions (for planning, maintenance analytics, documentation, etc.) [12] [11]. Understanding these embedded intentions is key to building AI agents that can collaborate in coding, since the agent needs to align with the user's implicit goals, not just follow literal instructions.

Modelling Intention Trees and Hierarchies

Intention trees refer to structured, hierarchical representations of goals or intentions. This concept is long-established in both AI planning and software design, and it's seeing renewed interest with LLM-based agents:

- **Goal Decomposition in Requirements:** As mentioned, requirement engineers use intention (goal) trees to break down high-level objectives into sub-goals. The I-Tree model is one example, where **intentions are nodes in a tree with decomposition relationships** [18]. High-level intentions (overall functionalities) get refined into more concrete sub-intentions (features or tasks), potentially across multiple levels. Such trees often include *constraints* or *relationships* (e.g., some goals might conflict or some must all be achieved). This hierarchical modeling is important because it mirrors how complex software is built to satisfy layered intentions. Older frameworks like KAOS and i *in the 1990s were foundational, introducing concepts like* AND/OR goal refinement *and* actor goals*. Today's I-Tree and related models build on that, providing meta-models for software teams to formally capture "what are we trying to achieve" in a tree form [18]. These intention trees can then guide development or be used to trace code back to stakeholder intents.

- **Hierarchical Plans in AI and HCI:** In AI, understanding a human or another agent's intent often naturally leads to a tree or hierarchy as well. For example, in **plan recognition** or **human-robot collaboration**, an agent might maintain a **hierarchical intention model** of the human's goals. Recent research on Hierarchical Intention Tracking (HIT) in robotics explicitly represents the human's intentions as a tree across multiple levels [20]. Each top-level intention (say a task in an assembly operation) can have several sub-intentions (subtasks), which in turn might break down further. The tracking algorithm then updates beliefs over this tree as the human's actions unfold, allowing the robot to adapt in real time. In *Figure 5* of one HIT study, they show an intention hierarchy tree where each child intention has exactly one parent, forming a strict tree structure

[21] [22]. By propagating probabilities down and up this tree, the robot can infer which subtask the human is likely addressing and adjust its assistance accordingly. This is a direct use of an **intention tree as a mental model** for an AI collaborating with humans.

- **LLM Agents and Task Trees:** When it comes to LLM-driven agents (like coding agents or general problem-solving agents), researchers are now incorporating intention hierarchies to manage complex tasks. The ReAcTree framework we discussed is a prime example of **modeling an agent's intentions as a tree** in real-time. In ReAcTree, each agent node has its own subgoal/intention, and if it determines that subgoal is too complex, it can spawn child agents with more granular subgoals [3]. The result is a dynamic tree of intentions where internal nodes break tasks into sub-tasks, and leaf nodes finally execute actions. Notably, ReAcTree introduces *control flow nodes* (like sequence, fallback, parallel) between sub-intentions, akin to behavior trees in game AI [23] [24]. This adds logic on how multiple intentions should be orchestrated. Such modeling is crucial for **long-horizon planning** – instead of a flat list of steps, the agent has a structured plan that can adapt. This idea of an "intention tree" is effectively bringing classical hierarchical planning (HTN, behavior trees) into the LLM era, where the LLM both *generates* and *navigates* the tree.

- **Intent Trees in Problem-Solving Dialogues:** Another context is interactive problem solving or conversational agents that manage complex goals over multiple sessions. A recent benchmark called *SessionIntentBench* (2025) introduces the notion of an **intention tree for e-commerce sessions**: essentially, modeling how a user's high-level purchase goal breaks into browsing sub-intents across sessions. They construct an intention tree from a session's events, labeling predicted user intentions at each branch [25]. This helps evaluate how well an LLM can track shifting or nested user goals over time. Such applications highlight that intention trees are not only for *pre-planned* design, but can be *discovered or inferred* from data by AI, especially with unsupervised or open-world intent discovery using LLMs.

- **From User Instructions to Trees:** In the specific case of *coding agents*, the ultimate aim of extracting implicit intentions is to create a sort of **tree of subtasks that the agent should perform**. For example, if a user says, "Implement feature X using Y library," the agent might break this into sub-intentions: (1) update dependencies to include Y, (2) write code for feature X (with its own sub-parts), (3) adjust tests or docs, etc. While we have few published papers *directly* on intention tree extraction from coding prompts, we see pieces of the puzzle in current research. SpecRover (NUS, 2024) and AdverIntent-Agent (CMU, 2025) both implicitly deal with branching intentions. **SpecRover** uses an LLM agent to infer specifications (intents) of various components as it navigates a codebase to fix an issue [26] [27]. One can view the outcome as a set of inferred intents (specs) for different functions that together solve the problem – essentially mapping a high-level issue into lower-level intentions. **AdverIntent-Agent** goes even further by *explicitly generating alternative intent hypotheses*: it doesn't assume a single intended program behavior, but rather creates multiple *candidate* intentions for how a buggy function is supposed to behave [28]. These multiple intents form a sort of *intent tree or forest* of possibilities – and by generating tests for each and seeing which one holds up, the agent homes in on the correct intention [29] [30]. This approach of *inferring a space of possible intentions and exploring them* is highly novel. It yields a "rich suggestion package" of program intents, test cases, and patches, ultimately aligning the final fix with the **developer's true intent** better than single-guess approaches [31] [30].

In conclusion, **intention trees serve as a powerful abstraction for modeling and reasoning about goals in a hierarchical way**, and they appear in many guises in recent research. Whether it's a requirements engineer mapping out stakeholder goals, a collaboration algorithm tracking a partner's

task hierarchy, or an autonomous coding agent breaking down a complex issue, the tree model helps to manage complexity and maintain clarity of purpose at each level. High-impact recent work has demonstrated that combining LLMs with intention tree modeling yields substantial benefits – from more reliable code generation (by planning out the intention before coding) [5], to better program repair (by considering multiple intent hypotheses) [32] [30], to improved long-horizon task success (by hierarchical subgoal planning) [3]. This synergy of LLMs with explicit **intentional structure** is paving the way for AI that can truly understand and execute our intentions, rather than just surface-level instructions.

Sources:

- Arora *et al.*, "Intent Detection in the Age of LLMs" (EMNLP 2024) – using chain-of-thought LLM prompting for dialogue intent classification [1].
- Xu *et al.*, "Your Coding Intent is Secretly in the Context..." (ArXiv 2025) – inferring developer intent from code context to improve LLM code completion [5] [16].
- Heričko *et al.*, "Commit-Level Software Change Intent Classification..." (Mathematics, 2024) – classifying commit intents (maintenance types) using CodeBERT on code diffs [11] [9].
- Wu *et al.*, "InferROI: LLM-based Resource-Oriented Intention Inference" (ISSTA 2024) – LLM infers resource acquisition/release intents in code for leak detection [7].
- Ruan *et al.*, "SpecRover: Code Intent Extraction via LLMs" (ArXiv 2024) – LLM agent iteratively infers specifications (intents) from project structure to guide bug fixes [26].
- Ye *et al.*, "AdverIntent-Agent: Adversarial Reasoning for Repair Based on Inferred Program Intent" (ISSTA 2025) – multi-agent system inferring multiple program intents and using adversarial tests to align patches with true intent [32] [30].
- Choi *et al.*, "ReAcTree: Hierarchical LLM Agent Trees for Task Planning" (ArXiv 2025) – LLM creates a hierarchical tree of subgoals with control flow for complex tasks [3].
- Tu *et al.*, "Domain Priori Knowledge based Integrated Solution Design..." – introduces I-Tree model for intention decomposition in requirements engineering [18].
- Zhou *et al.*, "Hierarchical Intention Tracking with Switching Trees" (ArXiv 2025) – modeling human intentions as a hierarchy (intention tree) for human-robot collaboration [20].

---

[1] [2] [2410.01627] Intent Detection in the Age of LLMs
https://arxiv.org/abs/2410.01627

[3] [4] [23] [24] ReAcTree: Hierarchical LLM Agent Trees with Control Flow for Long-Horizon Task Planning
https://arxiv.org/html/2511.02424v1

[5] [6] [16] [17] Your Coding Intent is Secretly in the Context and You Should Deliberately Infer It Before Completion
https://arxiv.org/html/2508.09537v1

[7] [8] LLM-based Resource-Oriented Intention Inference for Static Resource Detection
https://arxiv.org/html/2311.04448v2

[9] [10] [11] [12] [13] [14] [15] [19] Commit-Level Software Change Intent Classification Using a Pre-Trained Transformer-Based Code Model
https://www.mdpi.com/2227-7390/12/7/1012

[18] Meta-model of intention tree (I-Tree) | Download Scientific Diagram
https://www.researchgate.net/figure/Meta-model-of-intention-tree-I-Tree_fig1_344149995

[20] [21] [22] Hierarchical Intention Tracking with Switching Trees for Real-Time Adaptation to Dynamic Human Intentions during Collaboration
https://arxiv.org/html/2506.07004v1

[25] SessionIntentBench: A Multi-task Inter-session Intention-shift ... - arXiv
https://arxiv.org/html/2507.20185v1

[26] [27] SpecRover: Code Intent Extraction via LLMs *Joint first authors, ordered alphabetically.
https://arxiv.org/html/2408.02232v1

[28] [29] [30] [31] [32] [Literature Review] Adversarial Reasoning for Repair Based on Inferred Program Intent
https://www.themoonlight.io/en/review/adversarial-reasoning-for-repair-based-on-inferred-program-intent